

Parte 1:

1.1 R: Um valor muito alto pode "mentir" a média pois sobe muito o valor que realmente seria. Exemplo: em uma empresa, o salário do funcionario1 = (2.000), funcionario2 = (2.500), funcionario3 = (37.000) | $2.000 + 2.500 + 37.000 = 41.500/3 = 13.833,333$.

Desta forma uma média teve seu valor muito mais alto do realmente deveria.

1.2 R: Ignoram valores muito altos ou baixos, para não alterar os dados drasticamente, colocando dados da mesma escala (dados comparáveis).

Assim obtendo um resultado mais sensato e realista.

1.3 R: A margem de tolerância é um limite de erro aceitável estipulado pelo auditor, que se for excedido pode demonstrar erros ou fraudes nos valores.

Parte 2:

```
Limite de Segurança inicial: R$ 10,000.00
Digite o valor da venda 1 do dia (ex: 1234.56): 5000
Digite o valor da venda 2 do dia (ex: 1234.56): 6545
Digite o valor da venda 3 do dia (ex: 1234.56): 51612
Venda 1: R$ 5,000.00
Venda 2: R$ 6,545.00
Venda 3: R$ 51,612.00

--- Análise de Vendas ---
Média das vendas: R$ 21,052.33
ALERTA: SISTEMA EM QUARENTENA!
Sugestão: Vendas individuais muito altas. Considere ajustar o LIMITE_SEGURANCA se forem legítimas.

--- Verificação de Tipos de Dados ---
Tipo de LIMITE_SEGURANCA: <class 'float'>
Tipo de venda1: <class 'float'>
Tipo de venda2: <class 'float'>
Tipo de venda3: <class 'float'>
Tipo da média das vendas: <class 'float'>

O LIMITE_SEGURANCA atual é: R$ 10,000.00

# 1. Configuração Global
LIMITE_SEGURANCA = 10000.00
```

```

print(f"Limite de Segurança inicial: R$ {LIMITE_SEGURANCA:,.2f}")

# 2. Entrada de Dados e Casting
# Certifica que o programa aceite números decimais e substitui ',' por '.'
para conversão correta.
vendas = []
for i in range(1, 4):
    while True:
        try:
            valor_venda = input(f"Digite o valor da venda {i} do dia (ex:
1234.56): ").replace(',', '.')
            vendas.append(float(valor_venda))
            break
        except ValueError:
            print("Entrada inválida. Por favor, digite um número.")

venda1, venda2, venda3 = vendas[0], vendas[1], vendas[2]

print(f"Venda 1: R$ {venda1:,.2f}")
print(f"Venda 2: R$ {venda2:,.2f}")
print(f"Venda 3: R$ {venda3:,.2f}")

# 3. Processamento Interno (Função analisar_vendas) e Lógica de Investigação
def analisar_vendas(v1, v2, v3):
    global LIMITE_SEGURANCA # Permite a alteração da variável global

    print("\n--- Análise de Vendas ---")

    # Calcula a Média Simples
    media = (v1 + v2 + v3) / 3
    print(f"Média das vendas: R$ {media:,.2f}")

    # Lógica de Investigação
    if media > LIMITE_SEGURANCA:
        print("ALERTA: SISTEMA EM QUARENTENA!")
        # Exemplo de como o gerente poderia ajustar o limite se uma venda for
legítima
        # Isso aqui é um exemplo e pode ser ajustado com mais lógica ou
interação do usuário
        if any(v > LIMITE_SEGURANCA * 1.5 for v in [v1, v2, v3]): # Se alguma
venda for 50% acima do limite
            print("Sugestão: Vendas individuais muito altas. Considere ajustar
o LIMITE_SEGURANCA se forem legítimas.")
            # Neste ponto, um gerente poderia intervir para, por exemplo,
aumentar o LIMITE_SEGURANCA

```

```

        # Para este exemplo, não faremos uma alteração automática sem
input do usuário.

    discrepancia_detectada = False
    for venda in [v1, v2, v3]:
        if venda >= media * 5: # Se uma venda for 5x maior que a média
            print(f"ALERTA: Venda de R$ {venda:,.2f} é 5x maior que a média.
Sugere-se REVISÃO MANUAL!")
            discrepancia_detectada = True

    if not discrepancia_detectada and media <= LIMITE_SEGURANCA:
        print("Nenhuma anomalia grave detectada. Vendas dentro dos
parâmetros.")

    return media

# Chamada da função
media_vendas = analisar_vendas(venda1, venda2, venda3)

# 4. Apresentação: Exibir tipo de dado de cada variável
print("\n--- Verificação de Tipos de Dados ---")
print(f"Tipo de LIMITE_SEGURANCA: {type(LIMITE_SEGURANCA)}")
print(f"Tipo de venda1: {type(venda1)}")
print(f"Tipo de venda2: {type(venda2)}")
print(f"Tipo de venda3: {type(venda3)}")
print(f"Tipo da média das vendas: {type(media_vendas)}")

print(f"\nO LIMITE_SEGURANCA atual é: R$ {LIMITE_SEGURANCA:,.2f}") # Exibe o
limite atual, caso tenha sido 'considerado' para ajuste no futuro

```

Parte 3:

1.1 R: Ao processar esses valores, a média resultante foi de R\$ 1.766,66.

Resultado do Sistema: O código não ativou a "Quarentena", pois a média ficou abaixo dos R\$ 10.000,00 originais.

Análise de Decisão: O sistema se comportou exatamente como programado, mas revelou uma "brecha" de negócio: a venda de R\$ 5.000,00, apesar de ser quase 3x a média, não atingiu o gatilho de 5x para a "Revisão Manual".

Ajuste Lógico: Para maior sensibilidade em bancos reais, a linha `if venda >= media * 5` deveria ser recalibrada para um multiplicador menor (ex: 2.5x) para capturar anomalias mais sutis

1.2: R: No contexto de Engenharia de Software Bancário, o uso de global

`LIMITE_SEGURANCA` é considerado uma vulnerabilidade de segurança:

Exposição: Variáveis globais residem na memória compartilhada. Qualquer função mal escrita ou um "script injetado" pode alterar o limite para um valor infinito, permitindo que transações fraudulentas passem sem nenhum alerta.

Boas Práticas: Em sistemas reais, esse limite deve ser encapsulado (dentro de uma classe privada) ou buscado em um serviço externo de configuração com autenticação, garantindo que "quem processa" não tenha o poder de "mudar a regra" sem rastro de auditoria.

1.3 R: erro mais frustrante e comum nesta etapa é o `IndentationError` ou o `UnboundLocalError`.

O Erro: Tentar modificar o limite dentro da função esquecendo a linha global `LIMITE_SEGURANCA`.

A Intuição: Você percebe que o código "trava" visualmente. O Python parece ignorar o valor que você vê na tela. Antes de ler o erro, você nota que o bloco de código dentro do `if` parece "desalinhado" ou que a cor da variável mudou no editor.

Expectativa: "O computador sabe que estou falando daquele limite lá no topo."

Realidade: "Você está tentando criar uma nova regra aqui dentro, mas está tentando usar o valor dela antes mesmo de me dizer qual é."

